



1. How do you retrieve all the records from a table?

- To retrieve all the records from a table, you use the `SELECT * FROM table_name;` statement. The asterisk (*) is a wildcard character that selects all columns from the specified table.

2. How can you find the number of records in a table?

- You can find the number of records in a table using the `SELECT COUNT(*) FROM table_name;` query. The `COUNT(*)` function counts the total number of rows in the table.

3. How do you select records with a specific condition?

- To select records based on a specific condition, you use the `WHERE` clause, for example, `SELECT * FROM table_name WHERE condition;`. The condition could be a comparison or a combination of multiple conditions.

4. How do you retrieve unique values from a column?

- You can retrieve unique values from a column using the `DISTINCT` keyword, as in `SELECT DISTINCT column_name FROM table_name;`. This query filters out duplicate values in the specified column.

5. How do you sort records in ascending or descending order?

- To sort records in ascending order, you use `ORDER BY column_name ASC;`. For descending order, you use `ORDER BY column_name DESC;`. The `ASC` keyword is optional as it is the default order.

6. How do you find the maximum value in a column?

- You can find the maximum value in a column using the `MAX` function: `SELECT MAX(column_name) FROM table_name;`. This function returns the highest value in the specified column.

7. How do you find the minimum value in a column?

- Similarly, to find the minimum value, you use the `MIN` function: `SELECT MIN(column_name) FROM table_name;`. This query returns the lowest value in the column.

8. How do you calculate the average value of a column?

- The `AVG` function is used to calculate the average value: `SELECT AVG(column_name) FROM table_name;`. This function computes the mean value of the numeric data in the column.

9. How do you calculate the sum of a column?

- You calculate the sum of a column using the `SUM` function: `SELECT SUM(column_name) FROM table_name;`. This function adds up all the values in the specified column.



Intermediate SQL Questions

10. How do you join two tables and retrieve matching records?

To join two tables and retrieve matching records, you can use an **INNER JOIN**:

```
SELECT a.column1, b.column2
FROM table1 a
JOIN table2 b ON a.common_field = b.common_field;
```

- This query selects records where there is a match between the **common_field** in both tables.

11. How do you retrieve records that exist in one table but not in another using a **LEFT JOIN**?

You can retrieve such records using a **LEFT JOIN** and filtering with **WHERE**:

```
SELECT a.column1
FROM table1 a
LEFT JOIN table2 b ON a.common_field = b.common_field
WHERE b.common_field IS NULL;
```

- This query returns all records from **table1** that do not have matching records in **table2**.

12. How do you group records and apply aggregate functions?

You can group records using the **GROUP BY** clause and apply aggregate functions like **COUNT**, **SUM**, **AVG**, etc.:

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name;
```

- This query groups records by **column_name** and returns the count of records in each group.

13. How do you filter groups using the **HAVING** clause?

The **HAVING** clause is used to filter groups after aggregation:

```
SELECT column_name, COUNT(*)
```



```
FROM table_name  
GROUP BY column_name  
HAVING COUNT(*) > 1;
```

- This query returns only those groups where the count of records is greater than one.

14. How do you update records in a table?

To update records, you use the **UPDATE** statement with the **SET** clause:

```
UPDATE table_name  
SET column_name = value  
WHERE condition;
```

- This query updates the specified column with the new value where the condition is met.

15. How do you delete records from a table?

You can delete records using the **DELETE** statement with a **WHERE** clause:

```
DELETE FROM table_name  
WHERE condition;
```

- This query deletes records that meet the specified condition.

Advanced SQL Questions

16. How do you find the second highest salary in a table?

To find the second highest salary, you can use a subquery:

```
SELECT MAX(salary) AS SecondHighestSalary  
FROM table_name  
WHERE salary < (SELECT MAX(salary) FROM table_name);
```

- This query finds the maximum salary that is less than the highest salary.

17. How do you find employees who have a salary greater than the average salary?

You can find such employees using a subquery with the **AVG** function:

```
SELECT *  
FROM employees
```



```
WHERE salary > (SELECT AVG(salary) FROM employees);
```

- This query returns all employees whose salary is above the average.

18. How do you retrieve the top N records from a table?

To retrieve the top N records, you can use the **LIMIT** clause:

```
SELECT *  
FROM table_name  
ORDER BY column_name DESC  
LIMIT N;
```

- This query returns the top N records based on the specified order.

19. How do you find duplicate records in a table?

To find duplicate records, you group by the column and use **HAVING**:

```
SELECT column_name, COUNT(*)  
FROM table_name  
GROUP BY column_name  
HAVING COUNT(*) > 1;
```

- This query identifies records that appear more than once.

20. How do you create a new table from an existing table?

You can create a new table using the **CREATE TABLE AS** statement:

```
CREATE TABLE new_table AS  
SELECT column1, column2, column3  
FROM existing_table  
WHERE condition;
```

- This query creates a new table with selected columns and records.

21. How do you add a new column to an existing table?

To add a new column, you use the **ALTER TABLE** statement:

```
ALTER TABLE table_name  
ADD column_name data_type;
```

- This query adds a new column with the specified data type to the table.



22. How do you find the nth highest salary using `DENSE_RANK()`?

You can find the nth highest salary using the `DENSE_RANK()` window function:

```
SELECT salary
FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) as rank
    FROM employees
) as ranked_salaries
WHERE rank = n;
```

- This query ranks salaries and selects the one with the specified rank.

23. How do you use a common table expression (CTE) to simplify a query?

You can use a CTE to structure complex queries:

```
WITH cte AS (
    SELECT column_name, COUNT(*)
    FROM table_name
    GROUP BY column_name
)
SELECT *
FROM cte
WHERE COUNT > 1;
```

- This query simplifies the logic by using a temporary result set.

24. How do you write a query to transpose rows into columns?

Transposing rows into columns can be done using conditional aggregation:

```
SELECT
    MAX(CASE WHEN condition THEN column_name END) as column1,
    MAX(CASE WHEN condition THEN column_name END) as column2
FROM table_name
GROUP BY grouping_column;
```

- This query transforms rows into columns based on specific conditions.

25. How do you use window functions to calculate running totals?

You can calculate running totals with the `SUM()` function as a window function:

```
SELECT
```



```
column_name,  
SUM(column_name) OVER (ORDER BY column_name ROWS BETWEEN UNBOUNDED  
PRECEDING AND CURRENT ROW) as running_total  
FROM table_name;
```

- This query calculates a cumulative total for each row.

26. How do you write a query to find the longest consecutive sequence of values?

To find the longest consecutive sequence, you can use a CTE with `ROW_NUMBER()`:

```
WITH cte AS (  
    SELECT  
        value,  
        value - ROW_NUMBER() OVER (ORDER BY value) as grp  
    FROM table_name  
)  
SELECT value, COUNT(*)  
FROM cte
```

- GROUP BY grp;

Code with Swaroop